TACITVS

Protocol Security Audit Report

Sine ira et studio.
"WITHOUT ANGER OR BIAS."

Tacitus, Annals 1.1

TACITVS

Auctor Princeps

March 22, 2026

Contents

1 Table of Contents **2**

2 Protocol Summary **2**

3 Disclaimer **2**

4 Risk Classification **2**

5 Audit Details **3**

5.1 Commit Hash . . . 3

5.2 Scope . . . 3

5.3 Roles . . . 3

6 Executive Summary **3**

6.1 Issues Found . . . 4

7 Findings **4**

7.1 High . . . 4

7.2 Medium . . . 4

7.3 Low . . . 4

7.3.1 L-01: `setMinBidUsdValue()` allows mutation of bid admissibility during active auctions . . . 4

7.3.2 L-02: `_onAuctionEnd` performs contract-global LINK settlement rather than auction-scoped . . . 5

7.3.3 L-03: `isValidSignature()` and `bid()` enforce non-equivalent minimum fill constraints . . . 6

7.3.4 L-04: `transmit()` assumes monotonic `observationsTimestamp` but does not enforce upper bound . . . 7

7.3.5 L-05: `performUpkeep()` does not re-enforce the auction-end predicate before destructive settlement . . . 8

7.4 Informational . . . 9

Proof & Connect **9**

Prepared by: TACITVS Lead Security Researcher: - I.Leskov(TACITVS)

I. Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Commit Hash
  - Scope
  - Roles
- Executive Summary
  - Issues Found
- Findings
  - High
  - Medium
  - Low
  - Informational

II. Protocol Summary

Chainlink Payment Abstraction V2 is an auction-based system for converting accumulated fee tokens (ERC-20) into LINK via Dutch auction and CowSwap solver integration. The protocol manages concurrent auctions per asset with configurable parameters, price feeds from Chainlink Data Streams, and Chainlink Automation-driven lifecycle management.

III. Disclaimer

The TACITVS team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

IV. Risk Classification

Likelihood Impact	High	Medium	Low
High	H	H/M	M

Likelihood Impact	High	Medium	Low
Medium	H/M	M	M/L
Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

V. Audit Details

A. Commit Hash

```
2026-03-chainlink
```

B. Scope

```
BaseAuction.sol, GPV2CompatibleAuction.sol, PriceManager.sol  auction
   lifecycle, bid validation, settlement, price feed integration
```

C. Roles

- **ASSET_ADMIN_ROLE** Timelock-controlled admin for auction configuration
- **AUCTION_WORKER_ROLE** WorkflowRouter for Chainlink Automation upkeep
- **PRICE_ADMIN_ROLE** Trusted transmitter for Data Streams price reports
- **CowSwap Solvers** External solvers filling orders via GPV2 signature validation

VI. Executive Summary

Objective: Independent security review of auction mechanics, bid validation, settlement logic, and price feed integration in the Chainlink Payment Abstraction V2 system

Severity	Count
High	0
Medium	0
Low	5
Info	0
Total	**5**

Key Risk Areas: Mid-auction parameter mutation, cross-auction settlement coupling, inconsistent fill path constraints, price feed timestamp assumptions, keeper performData trust

Top 3 Recommendations: 1. Guard `setMinBidUsdValue` with `_whenNoLiveAuctions()` 2. Add upper-bound check on `observationsTimestamp` 3. Re-enforce auction-end predicate in `performUpkeep`

A. Issues Found

Severity	Number of issues found
High	0
Medium	0
Low	5
Info	0
Total	5

VII. Findings

A. High

No high severity vulnerabilities identified.

B. Medium

No medium severity vulnerabilities identified.

C. Low

I. L-01: `setMinBidUsdValue()` allows mutation of bid admissibility during active auctions

Links: - BaseAuction.sol#L474-L486

Description:

All configuration setters that determine auction economics enforce `_whenNoLiveAuctions()` to ensure auction parameters remain static during the auction lifecycle:

Setter	Guard
`setAssetOut()`	`_whenNoLiveAuctions()`
`setAssetOutReceiver()`	`_whenNoLiveAuctions()`
`setFeeAggregator()`	`_whenNoLiveAuctions()`
`applyAssetParamsUpdates()`	per-asset live auction check
`applyFeedInfoUpdates()`	via `_onFeedInfoUpdate`

Setter	Guard
`setMinBidUsdValue()`	**None**

`_setMinBidUsdValue()` lacks this guard:

```
function _setMinBidUsdValue(uint88 minBidUsdValue) private {
    if (minBidUsdValue == 0) {
      revert Errors.InvalidZeroValue();
    }
    if (s_minBidUsdValue == minBidUsdValue) {
      revert Errors.ValueNotUpdated();
    }
    s_minBidUsdValue = minBidUsdValue;
    emit MinBidUsdValueSet(minBidUsdValue);
}
```

This breaks the invariant that **auction parameters determining bid eligibility remain static during the auction lifecycle**. An active bidder can prepare an otherwise valid bid that becomes invalid solely due to a mid-auction governance mutation of the minimum bid threshold. Conversely, lowering the threshold mid-auction admits bids that were ineligible when the auction started.

Impact:

Active auction participants cannot rely on bid eligibility remaining stable for the duration of the auction, even though other economic parameters are explicitly frozen while live auctions exist. The `ASSET_ADMIN_ROLE` is a Timelock (trusted).

Recommended Mitigation:

Add `_whenNoLiveAuctions()` to `_setMinBidUsdValue()`.

II. L-02: `_onAuctionEnd` performs contract-global LINK settlement rather than auction-scoped

Links: - BaseAuction.sol#L393-L396

Description:

When any auction ends, `_onAuctionEnd` settles the **entire** contract-global `assetOut` (LINK) balance rather than only the proceeds attributable to that specific auction:

```
uint256 assetOutBalance = IERC20(s_assetOut).balanceOf(address(this));
if (assetOutBalance > 0) {
    IERC20(s_assetOut).safeTransfer(s_assetOutReceiver, assetOutBalance);
}
```

This creates unexpected coupling between concurrent auctions. When auctions for WETH and USDC run concurrently:

1. Bidders on the WETH auction pay LINK LINK accumulates in the contract

2. The USDC auction ends (time expiry or dust condition)
3. `_onAuctionEnd(USDC)` sweeps **all** LINK from the contract - including LINK paid by WETH bidders
4. The WETH auction remains live, but its settlement proceeds have already been removed

The broken property: **ending one auction performs settlement over contract-global LINK state rather than auction-local proceeds**.

Impact:

Loss of auction isolation - concurrent auction closures have cross-auction side effects on LINK settlement. All LINK reaches the same `assetOutReceiver`, so no funds are permanently lost, but the lifecycle assumption that an auction closure only settles proceeds attributable to that auction does not hold.

Recommended Mitigation:

Document this behavior explicitly as a known design property. If auction-scoped accounting is desired, consider tracking per-auction LINK accumulation rather than using `balanceOf`.

III. L-03: `isValidSignature()` AND `bid()` ENFORCE NON-EQUIVALENT MINIMUM FILL CONSTRAINTS

Links: - GPV2CompatibleAuction.sol#L141 - BaseAuction.sol#L430-L434

Description:

The system exposes two fill paths with non-equivalent admissibility rules for the same economic action (filling an auction):

`bid()` enforces `s_minBidUsdValue`:

```
uint256 bidUsdValue = (amount * assetPrice) / (10 ** assetParams.decimals
    );
uint88 minBidUsdValue = s_minBidUsdValue;
if (bidUsdValue < minBidUsdValue) {
    revert BidValueTooLow(bidUsdValue, minBidUsdValue);
}
```

`isValidSignature()` only checks `order.sellAmount > 0`:

```
if (order.sellAmount == 0) {
    revert Errors.InvalidZeroAmount();
}
```

This mismatch means CowSwap solvers can fill orders for amounts below `minBidUsdValue`. The omission is undocumented.

Impact:

No economic damage due to rounding in protocols favor (`mulDivUp`/`mulWadUp`). However, the documentation gap may lead integrators or governance to misunderstand the enforceable minimum order size.

Recommended Mitigation:

Either document that solver-based fills are intentionally exempt from `minBidUsdValue`, or enforce equivalent minimum-size constraints across both fill paths.

IV. L-04: `transmit()` ASSUMES MONOTONIC `observationsTimestamp` BUT DOES NOT ENFORCE UPPER BOUND

Links: - PriceManager.sol#L162-L164

Description:

> **Note:** This finding is distinct from V12 F-3 (Expired reports accepted as valid prices), which addresses `validFromTimestamp` and `expiresAt` fields being ignored. This finding concerns a different field (`observationsTimestamp`) and a different property (missing upper bound).

The local freshness logic in `transmit()` assumes monotonic, non-future timestamps but does not enforce that assumption on-chain:

```
if (report.observationsTimestamp < block.timestamp - feedInfo.
    stalenessThreshold) {
    revert Errors.StaleFeedData();
}
```

There is no corresponding check that `observationsTimestamp <= block.timestamp`. A verified report with `observationsTimestamp = block.timestamp + 100_000` would:

1. Pass the staleness check (future timestamp is always `>= block.timestamp - stalenessThreshold`)
2. Be stored with the future timestamp
3. Remain fresh in `_getAssetPrice()` for `100_000 + stalenessThreshold` seconds instead of just `stalenessThreshold`

Impact:

Reports go through `i_streamsVerifierProxy.verifyBulk()` and `PRICE_ADMIN_ROLE` is trusted. However, the implementation relies on upstream timestamp correctness rather than enforcing a local non-future invariant.

Recommended Mitigation:

Add an upper bound check:

```
if (report.observationsTimestamp > block.timestamp) {
    revert InvalidFeedTimestamp();
}
```

V. L-05: performUpkeep() does not re-enforce the auction-end predicate before destructive settlement

Links: - BaseAuction.sol#L359-L369 - BaseAuction.sol#L246-L254

Description:

`performUpkeep()` should not assume that `endedAuctions` passed in `performData` was necessarily produced by a fresh and honest `checkUpkeep()` call. The end-of-auction predicate should be re-enforced on-chain before performing destructive settlement actions.

Currently, the ended-auctions loop only checks that the auction exists:

```
for (uint256 i; i < endedAuctions.length; ++i) {
    address asset = endedAuctions[i];
    if (s_auctionStarts[asset] == 0) {
        revert InvalidAuction(asset);
    }
    _onAuctionEnd(endedAuctions[i], hasFeeAggregator);
    delete s_auctionStarts[asset];
    emit AuctionEnded(asset);
}
```

Compare with `checkUpkeep`, which requires the auction-end predicate to hold:

```
if (
    auctionStart + assetParams.auctionDuration < block.timestamp
        || (isPriceValid && assetBalanceUsdValue < assetParams.
            minAuctionSizeUsd)
) {
    endedAuctions[endedAuctionsIdx++] = asset;
}
```

Neither condition is re-checked in `performUpkeep`. The destructive settlement actions that execute without re-validation: 1. Unsold auctioned assets returned to FeeAggregator 2. Entire LINK balance swept to `assetOutReceiver` 3. CowSwap vault relayer approval revoked, invalidating pending solver orders

Impact:

The `AUCTION_WORKER_ROLE` is trusted (WorkflowRouter). However, `performUpkeep` performs irreversible settlement without on-chain re-enforcement of the predicate that justifies those actions.

Recommended Mitigation:

Re-enforce the auction-end predicate inside the `performUpkeep` ended-auctions loop:

```
AssetParams memory assetParams = s_assetParams[asset];
uint256 auctionStart = s_auctionStarts[asset];
(uint256 assetPrice,,bool isPriceValid) = _getAssetPrice(asset, false);
uint256 assetBalance = IERC20(asset).balanceOf(address(this));
uint256 assetBalanceUsdValue = (assetBalance * assetPrice) / (10 **
    assetParams.decimals);

bool isExpired = auctionStart + assetParams.auctionDuration < block.
    timestamp;
```

```
bool isDust = isPriceValid && assetBalanceUsdValue < assetParams.
    minAuctionSizeUsd;

if (!isExpired && !isDust) {
     revert AuctionStillActive(asset);
}
```

D. Informational

No informational findings.

Proof & Connect

This report is cryptographically anchored on-chain. Scan the left QR to verify the exact SHA-256 fingerprint and blockchain transaction. Scan the right QR to explore my portfolio or book a call.

Scan to verify on-chain proof

Scan for portfolio & booking